

Manual Técnico – Proyecto2

Angel Emanuel Rodriguez Corado

202404856

Índice

Introducción	3
Objetivos.....	3
Público Objetivo	3
Requisitos técnicos.....	3
Hardware	3
Software	3
Lógica General del Sistema	4
Funcionamiento de los Módulos Principales	4
LexicalAnalyzer.ts	4
SyntacticAnalyzer.ts	5
Token.ts.....	6
Error.ts.....	7
Transpiler.ts	7
analyze.controller.ts.....	9
Index.ejs.....	10
Compilerandrun.ts	11
BNF.txt	12
index.js	13
index.ts	14
analyze.route.ts	15
Automata Finito Determinista AFD	16

Introducción

Este documento describe los componentes técnicos del proyecto desarrollado en TypeScript para analizar el lenguaje de C# , El código permite analizar a nivel sintactico y léxico y ver diferentes resultados. El sistema incluye un analizador léxico basado en un autómata finito determinista (AFD) junto con un analizador sintactico, generación de reportes de tokens, manejo de errores, traducción a Typescript y la salida en consola.

Objetivos

- Documentar la lógica de implementación del analizador léxico.
- Explicar la estructura del código fuente.
- Servir como guía técnica para mantenimiento o evolución del sistema.

Público Objetivo

Este manual está dirigido a desarrolladores, docentes y estudiantes que deseen entender o modificar la lógica del proyecto, o que necesiten extender su funcionalidad.

Requisitos técnicos

Hardware:

- Procesador Intel i5 o superior
- Mínimo 4 GB de RAM

Software:

- Node.js 18+
- TypeScript
- Express.js
- EJS
- Navegador web moderno

Lógica General del Sistema

1. El usuario carga un archivo .cs o edita el texto directamente en el editor.
2. Al presionar “Analizar”, el backend ejecuta un analizador léxico y sintáctico personalizado basado en AFD.
3. Se generan:
 - a. Tokens válidos → mostrados en tabla. o Errores léxicos y sintácticos → mostrados en su respectiva tabla
 - b. Salida Traducida → Se muestra la traducción de C# a Typescript
 - c. Salida en Consola → Se muestra la compilación del código

Funcionamiento de los Módulos Principales

LexicalAnalyzer.ts

- Implementa el Autómata Finito Determinista (AFD).
- Lee línea por línea y analiza cada carácter.
- Clasifica lexemas en:
 - Palabras reservadas
 - Cadenas (entre comillas)
 - Números (enteros)
 - Símbolos
- Registra tokens válidos y errores léxicos con fila y columna.
- **Se trabajan las palabras reservadas de forma que se analizan desde una lista comparándolas con la entrada en el frontend**

Link del código:

https://github.com/Deltai-cod12/Proyecto2_LFP_202404856.git

SyntacticAnalyzer.ts

- Implementa el Analizador Sintáctico Descendente.
- Recibe la lista de tokens generados por el análisis léxico.
- Valida que la estructura del código siga la gramática del lenguaje definido.
- Utiliza producciones como:
 - INICIO → USING_DECL CLASS_DECL
 - CLASS_DECL → public class IDENTIFIER { MAIN_DECL }
 - MAIN_DECL → static void Main (string[] args) { SENTENCIAS }
- Verifica el orden correcto de símbolos como llaves, paréntesis y punto y coma.
- Reporta errores sintácticos con fila y columna si falta un símbolo o palabra clave.
- Puede generar el árbol de derivación o lista de instrucciones si se requiere.
- Se detiene si encuentra un error grave que impide continuar el análisis.

Link del Código:

https://github.com/Deltai-cod12/Proyecto2_LFP_202404856.git

Token.ts

- Define interfaces para:
- Token: tipo, valor, fila, columna • Error: carácter, descripción, ubicación
- Enumera los tipos de tokens reconocidos.

```
export enum Type {
    UNKNOWN,
    KEY_O,
    KEY_C,
    BRA_O,
    BRA_C,
    PAR_O,
    PAR_C,
    SEMICOLON,
    COMMA,
    PERIOD,
    ASSIGN,
    PLUS,
    MINUS,
    MULT,
    DIV,
    INC,
    DEC,
    EQUAL,
    DIFF,
    LESS,
    GREATER,
    LESS_EQ,
    GREATER_EQ,
    IDENTIFIER,
    INTEGER,
    DECIMAL,
    COMMENT,
    MULTICOMMENT,
    STRING,
    CHAR,
    R_USING,
    R_SYSTEM,
    R_PUBLIC,
    R_CLASS,
    R_STATIC,
    R_VOID,
    R_MAIN,
    R_STRING,
    R_INT,
    R_FLOAT,
    R_CHAR,
    R_BOOL,
    R_FALSE,
    R_TRUE,
    R_CONSOLE,
    R_UNTILLINE,
    R_IF,
    R_ELSE,
    R_FOR
}

export class Token {
    private typeTokenString: string;
    private typeToken: Type;
    private lexeme: string;
    private row: number;
    private column: number;

    constructor(typeToken: Type, lexeme: string, row: number, column: number) {
        this.typeTokenString = typeToken;
        this.typeToken = typeToken;
        this.lexeme = lexeme;
        this.row = row;
        this.column = column;
    }

    getType(): Type {
        return this.typeToken;
    }

    getLexeme(): string {
        return this.lexeme;
    }

    getRow(): number {
        return this.row;
    }

    getColumn(): number {
        return this.column;
    }

    getTypeTokenString(): string {
        return this.typeTokenString;
    }
}
```

Error.ts

Recopila los errores sintacticos si es que hubieran durante el análisis

```
export class Error {  
    private row;  
    private column;  
    private lexeme: string;  
    private description: string;  
  
    constructor(lexeme: string, description: string, row: number, column: number) {  
        this.lexeme = lexeme;  
        this.description = description;  
        this.row = row;  
        this.column = column;  
    }  
  
    getLexeme(): string {  
        return this.lexeme;  
    }  
  
    getDescription(): string {  
        return this.description;  
    }  
  
    getRow(): number {  
        return this.row;  
    }  
  
    getColumn(): number {  
        return this.column;  
    }  
}
```

Transpiler.ts

- **Toma la lista de instrucciones o tokens válidos** (provenientes del SyntacticAnalyzer).
- **Traduce** esas instrucciones a otro lenguaje o formato de salida (por ejemplo, de un pseudolenguaje a C#, Python, etc.).
- **Construye el código equivalente** manteniendo la lógica del original.
- Puede incluir funciones para:
 - Generar el nuevo código línea por línea.
 - Formatear la sintaxis según el lenguaje destino.
 - Concatenar el resultado final como un string para mostrarlo o guardarlo.

- Se usa como parte del proceso de **compilación o interpretación cruzada**.

Link del Código:

https://github.com/Deltai-cod12/Proyecto2_LFP_202404856.git

analyze.controller.ts

```
import { Request, Response } from "express";
import { LexicalAnalyzer } from "../Analyzer/LexicalAnalyzer";
import { Token } from "../Analyzer/Token";
import { SyntacticAnalyzer } from "../Analyzer/SyntacticAnalyzer";
import { Error } from "../Analyzer/Error";
import { Transpiler } from "../Analyzer/Transpiler";
import { Instruction } from "../models/abstract/Instruction";
import { compileAndRun } from "../utils/compileAndRun";

export const home = (req: Request, res: Response) => {
  res.render('pages/index');
}

export const analyze = async (req: Request, res: Response) => {
  const body = req.body;

  const scanner: LexicalAnalyzer = new LexicalAnalyzer();
  const tokenList: Token[] = scanner.scanner(body);

  const parser = new SyntacticAnalyzer(tokenList);
  parser.parser();

  const errorParser: Error[] = parser.getErrors();

  let code = '';
  let output = '';

  if (errorParser.length === 0) {
    const transpiler = new Transpiler(tokenList);
    transpiler.parser();

    transpiler.getInstructions().forEach((instruction: Instruction) => {
      code += instruction.transpiler();
    });

    try {
      output = await compileAndRun(code);
    } catch (err) {
      output = `Error en compilación o ejecución: ${err}`;
    }
  }

  res.json({
    tokens: tokenList,
    errors: scanner.getErrorList(),
    syntacticErrors: errorParser,
    traduction: code,
    colors: scanner.getColors(),
    output: output
  });
}
```

Index.ejs

Pagina principal donde se muestra la tabla tokens

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.0.0/dist/css/bootstrap.min.css"
    integrity="sha384-Gn5384-xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm" crossorigin="anonymous">
  <link rel="stylesheet" href="css/index.css">
  <title>Proyecto</title>
</head>

<body>

  <header>
    <%- include('../partials/header'); %>
  </header>

  <div class="container_user">
    <div class="row justify-content-center">
      <div class="col">
        <h1>Editor de Texto</h1>
        <%- include('../partials/editor', {id: 'editor' , editable: true}); %>
      </div>
      <div class="col">
        <h1>Salida Traducida</h1>
        <%- include('../partials/editor', {id: 'salida' , editable: false}); %>
      </div>
      <div class="col">
        <h1>Salida por Consola</h1>
        <div id="console_output" class="editor no-editable"></div>
      </div>
    </div>
    <div class="row justify-content-center">
      <button id="analyze" class="btn btn-primary btn_user">
        Analizar
      </button>
    </div>
  </div>

  <div class="row">
    <div class="col">
      <h1>Tabla de Tokens</h1>
      <%- include('../partials/table', {id: 'tokens'}); %>
    </div>
    <div class="col">
      <h1>Errores Léxicos</h1>
      <%- include('../partials/table', {id: 'errors'}); %>
    </div>
    <div class="col">
      <h1>Errores Sintácticos</h1>
      <%- include('../partials/table', {id: 'syntacticErrors'}); %>
    </div>
  </div>
</div>

<script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"
  integrity="sha384-KJ3o2DKtIkvYIK3UEENzmM7KcKrR/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN"
  crossorigin="anonymous"></script>
<script src="https://cdn.jsdelivr.net/npm/popper.js@1.12.9/dist/umd/popper.min.js"
  integrity="sha384-ApNbgh9B+Y1QKt3Rn7W3mgPxm9/KcQsAP7hUibX39j7fakFPskvXusvfa0b4Q"
  crossorigin="anonymous"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@4.0.0/dist/js/bootstrap.min.js"
  integrity="sha384-JZR6Spejh4U02d8j0t6vLEHfe/JQGiRRSQQxSfFFWp11MuQvDyAjUar5+76PVCmY1"
  crossorigin="anonymous"></script>
<script src="js/index.js"></script>
</body>

</html>
```

Compilerandrun.ts

```
import { writeFileSync, unlinkSync } from 'fs';
import { exec } from 'child_process';
import path from 'path';
import os from 'os';

export async function compileAndRun(tsCode: string): Promise<string> {
  // Usar carpeta temporal del sistema
  const tempDir = os.tmpdir();
  const tsFile = path.join(tempDir, 'tempCode.ts');
  const jsFile = path.join(tempDir, 'tempCode.js');

  // Escribir el archivo temporal TypeScript
  writeFileSync(tsFile, tsCode, 'utf8');

  // Compilar el archivo con tsc
  await new Promise<void>((resolve, reject) => {
    exec(`npx tsc ${tsFile}`, (error, stdout, stderr) => {
      if (error) return reject(stderr || error.message);
      resolve();
    });
  });

  // Ejecutar el archivo JavaScript generado
  const output: string = await new Promise((resolve, reject) => {
    exec(`node ${jsFile}`, (error, stdout, stderr) => {
      if (error) return reject(stderr || error.message);
      resolve(stdout);
    });
  });

  // Eliminar los archivos temporales
  try {
    unlinkSync(tsFile);
    unlinkSync(jsFile);
  } catch (e) {
    console.warn('No se pudo borrar archivos temporales:', e);
  }

  return output;
}
```

Gramatica del lenguaje que estamos analizando

12

index.js

Se encarga de la carga del archivo, al igual que hacer la limpieza y la acción del botón analizar

Este script maneja la interacción del usuario con la interfaz. Sus funciones principales son:

- **Cargar archivo .pkfp:** Lee el contenido del archivo seleccionado y lo coloca en el área de texto.
- **Limpiar el editor:** Borra el contenido del área de texto.
- **Guardar archivo:** Descarga el contenido actual del editor como un archivo .pkfp.
- **Análisis léxico:**
 - Envía el texto al backend (/analyze) para su análisis. o Muestra los tokens válidos en una tabla.
 - En caso de errores, los envía al servidor para mostrar el reporte HTML.

[illegible]

index.ts

Configura y lanza el servidor express al igual que define middelwares estáticos y motor de vistas

```
import express from 'express';
import analyzeRouter from './routes/analyze.route';

const app = express();

app.set('view engine', 'ejs');

app.use(express.static('public'));
app.use(express.text());

app.use(analyzeRouter);

app.listen(3000, () => {
  console.log("The server is listening on: http://localhost:3000");
});
```

analyze.route.ts

Recibe el texto para el análisis, renderiza el index.ejs y errores.ejs

```
import { Router } from "express";
import { analyze, home } from "../controllers/analyze.controller";

const analyzeRouter = Router();

analyzeRouter.get('/', home);
analyzeRouter.post('/analyze', analyze);

export default analyzeRouter;
```

Automata Finito Determinista AFD

